

SKILL 18

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

typedef struct {
    int job_id;
    pid_t pid;
    char status[20];
} Job;

Job job;

/* Signal handler */
void signal_handler(int sig) {
    if (sig == SIGUSR1) {
        printf("Child received SIGUSR1\n");
    }
}

void show_job() {
    printf("Job ID: %d | PID: %d | Status: %s\n",
        job.job_id,
        job.pid,
        job.status);
}

int main() {
```

```
int main() {
    printf("=== Task 18: Signals and Job Recovery ===\n");

    printf("\n=== Signal Mask Test ===\n");

    sigset_t set;

    sigemptyset(&set);
    sigaddset(&set, SIGUSR1);

    sigprocmask(SIG_BLOCK, &set, NULL);

    printf("SIGUSR1 blocked temporarily\n");

    signal(SIGUSR1, signal_handler);

    printf("\n=== Create Child Process ===\n");

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return 1;
    }

    if (pid == 0) {
        printf("Child PID: %d\n", getpid());
        printf("Process Group ID: %d\n", getpgrp());

        sigprocmask(SIG_UNBLOCK, &set, NULL);

        printf("SIGUSR1 unblocked in child\n");

        sleep(10);

        printf("Child completed normally\n");

        exit(0);
    }

    job.job_id = 1;
    job.pid = pid;
    strcpy(job.status, "Running");
}
```

```
if (pid == 0) {
    printf("Child PID: %d\n", getpid());
    printf("Process Group ID: %d\n", getpgrp());

    sigprocmask(SIG_UNBLOCK, &set, NULL);

    printf("SIGUSR1 unblocked in child\n");

    sleep(10);

    printf("Child completed normally\n");

    exit(0);
}

job.job_id = 1;
job.pid = pid;
strcpy(job.status, "Running");

printf("\n=== Job Table ===\n");
show_job();

sleep(1);

printf("\n=== Signal Delivery Test ===\n");

kill(pid, SIGUSR1);

printf("SIGUSR1 sent to child\n");

sleep(1);

printf("\n=== Stop Job Test ===\n");

kill(pid, SIGSTOP);

strcpy(job.status, "Stopped");

show_job();

sleep(2);

printf("\n=== Resume Stopped Job ===\n");
```

```

kill(pid, SIGCONT);

strcpy(job.status, "Running");

printf("SIGCONT sent to child\n");

show_job();

printf("\n=== Background Recovery Test ===\n");

printf("Job resumed successfully in background\n");

printf("\nWaiting for child to finish...\n");

waitpid(pid, NULL, 0);

strcpy(job.status, "Completed");

printf("\n=== Updated Job Status ===\n");

show_job();

printf("\n=== Signal Behavior Summary ===\n");
printf("Signal delivery tested\n");
printf("Signal mask tested\n");
printf("Process group reviewed\n");
printf("SIGSTOP tested\n");
printf("SIGCONT tested\n");
printf("Job table updated\n");
printf("Background recovery verified\n");

printf("\n=== Task 18 Complete ===\n");

return 0;
}

```

Output:

```
=== Task 18: Signals and Job Recovery ===
```

```
=== Signal Mask Test ===
```

```
SIGUSR1 blocked temporarily
```

```
=== Create Child Process ===
```

```
=== Job Table ===
```

```
Child PID: 659
```

```
Job ID: 1 | PID: 659 | Status: Running
```

```
Process Group ID: 658
```

```
SIGUSR1 unblocked in child
```

```
=== Signal Delivery Test ===
```

```
SIGUSR1 sent to child
```

```
Child received SIGUSR1
```

```
Child completed normally
```

```
=== Stop Job Test ===
```

```
Job ID: 1 | PID: 659 | Status: Stopped
```

```
=== Resume Stopped Job ===
```

```
SIGCONT sent to child
```

```
Job ID: 1 | PID: 659 | Status: Running
```

```
=== Background Recovery Test ===
```

```
Job resumed successfully in background
```

```
Waiting for child to finish...
```

```
=== Background Recovery Test ===  
Job resumed successfully in background  
  
Waiting for child to finish...  
  
=== Updated Job Status ===  
Job ID: 1 | PID: 659 | Status: Completed  
  
=== Signal Behavior Summary ===  
Signal delivery tested  
Signal mask tested  
Process group reviewed  
SIGSTOP tested  
SIGCONT tested  
Job table updated  
Background recovery verified
```